

PAGE 1

Rapport du projet robot LU2IN013
Groupe BoysBand

[insérer image robot?]

Membres du groupe:

- Alex Argeles
- Mateusz Wieckowski
- Xiangkun Zhao
- Rayane Nom?
- Kevin Tratat

PAGE 2

Table des matières

- Introduction
- Mode d'emploi
- Schéma fonctionnel
- Schéma d'architecture (UML)
- Équations régissant les mouvements du robot en simulation
- Pistes d'améliorations
- Conclusion

PAGE 3

Introduction:

Le projet porte sur la conception d'un contrôleur de robot mobile à deux roues motrices. L'objectif principal était de développer une architecture modulaire permettant de switcher entre un simulateur Python (basé sur Pygame) et un robot physique (EasyGoPiGo3) sans modifier le code de stratégie.

Mode d'emploi:

Prérequis : Pygame et dépendances du robot (easygopigo3)

Accéder au github du projet : <https://github.com/CrazyDreamer2000/ProjetRobotique-UL2IN013>, puis dans la branche "Branche-Master", télécharger le code en cliquant sur "Download ZIP".

Robot simulation:

Pour exécuter la simulation, rendez vous dans le répertoire "Projet_Clean" et tapez en ligne de commande "python [main.py](#)". Une interface graphique apparaîtra mettant à l'image le robot ainsi que des obstacles pré-générés. La stratégie par défaut qui se lancera sur le tracage de polygone. Pour le changer, à la suite de "python [main.py](#)", ajoutez "--algo "avancer"" pour que le robot ne fasse qu'avancer. La liste des algorithmes est disponible dans le fichier "__init__.py" du dossier "controle". De plus, un clique-gauche sur la fenêtre graphique fera apparaître au même endroit un obstacle carré.

Le fichier config.txt permet de modifier les dimensions du robot et du monde

Robot Réel:

On accède au robot réel en se connectant à son réseau wifi avec le mot de passe GOPIGO2IN013, puis par ssh avec la commande « ssh pi@192.168.13.1 » puis en rentrant le mot de passe pi.

Ensuite, on transfère notre code au robot avec « scp -r Projet_Clean pi@192.168.13.1: ».

Ensuite, à partir du répertoire "Projet_Clean", on exécute le fichier [main.py](#). On peut changer de stratégie de la même manière qu'en simulation.

PAGE 4:

[schéma fonctionnel]

[schéma UML]

Équations du Mouvement et de la Cinématique:

Le simulateur repose sur un modèle de **cinématique différentielle** pour un robot à deux roues motrices indépendantes.

1. Paramètres Physiques

Les calculs utilisent les constantes définies dans la configuration du robot :

- R : Rayon des roues (33.25 mm, soit 66.5/2).
- L : Écartement entre les roues (Entraxe = 117 mm).

2. Calcul des Vitesses (Modèle Différentiel)

À partir des vitesses de rotation de la roue gauche (ω_g) et de la roue droite (ω_d) en rad/s, on calcule la vitesse linéaire de chaque roue :

- $v_g = \omega_g \times R$
- $v_d = \omega_d \times R$

La **vitesse linéaire du centre du robot** (v) et sa **vitesse angulaire** (ω) sont alors définies par :

- $v = 2v_g + v_d$
- $\omega = L v_d - v_g$

3. Intégration de la Posture (Mise à jour de la position)

À chaque pas de temps dt (calculé localement entre deux appels), la nouvelle position (x,y) et l'orientation θ du robot dans le monde sont mises à jour selon les équations suivantes :

- $x_{t+dt} = x_t + v \cdot \cos(\theta_t) \cdot dt$
- $y_{t+dt} = y_t + v \cdot \sin(\theta_t) \cdot dt$
- $\theta_{t+dt} = \text{normaliser}(\theta_t + \omega \cdot dt)$

4. Transformation de Repère (Local vers Monde)

Pour l'affichage et la détection de collisions (SAT), les points du robot définis dans son repère local sont transformés en coordonnées "monde" :

- $x_{\text{monde}} = x_{\text{local}} \cdot \cos(\theta) - y_{\text{local}} \cdot \sin(\theta) + x_{\text{robot}}$
- $y_{\text{monde}} = x_{\text{local}} \cdot \sin(\theta) + y_{\text{local}} \cdot \cos(\theta) + y_{\text{robot}}$

5. Gestion des Angles

L'orientation du robot est systématiquement ramenée dans l'intervalle $[-\pi, \pi]$ pour éviter les dérives de calcul via la fonction de normalisation : $\text{Angle_Normalise}' = (a + \pi) \pmod{2\pi} - \pi$

PAGE 5:

Pistes d'amélioration:

Conclusion: